

Operation — Broken Pipeline

CI/CD → registry → Kubernetes supply-chain · Ref SSOU-WR-2026-1006-BP · LAB ONLY · TLP:AMBER

Eswar Mahalingam
SSOU / Adversary Sim · Cand.
13
GS-STU-DSOU-2026-039A

Trust chain not one container	New digest unsigned	cluster-admin deploy SA	IRSA→cloud secret read	16/16 tests pass	LAB ONLY no exploit code
---	-------------------------------	-----------------------------------	----------------------------------	----------------------------	------------------------------------

Thesis — the dangerous move is compromising the CI/CD trust chain so malicious content ships inside a trusted pipeline: stale developer identity → off-hours run → same tag / new unsigned digest → deploy → cluster-admin service account → cloud. Detection-and-analysis only, built on synthetic audit data. **Trust the pipeline, verify the artifact, hunt the identity.**

Mission 01 — Kubernetes attack-surface map

DEVELOPER (ex.developer – terminated, still in automation)

- └ GITLAB self-hosted CI, off-hours run outside the release window
 - └ RUNNER can reach the container registry + the cluster deploy SA
 - └ REGISTRY registry.corp.internal/payments/api – tag :latest reused, NEW digest sha256:beef9999..., NO cosign signature / provenance
 - └ KUBERNETES payments ns deployment patched to the bad digest; deployer SA = cluster-admin (excessive RBAC) → lists secrets, creates a kube-system pod (hostPath)
 - └ CLOUD pod workload identity (IRSA) → AssumeRole → GetSecretValue

Surface element	What reveals it (audit-only)
Exposed GitLab/CI	GitLab audit auth.success / pipeline.run by a terminated actor from an unusual host
Registry metadata	image.push rows: tag, digest , signature/provenance flags
K8s namespaces/SAs	audit user.username = system:serviceaccount:payments:deployer, target namespaces
High-value CI/CD components	the deploy SA token; the registry push credential; the runner
Excessive RBAC	payments:deployer bound to cluster-admin (intended: patch deployments in one ns)
Evidence of unexpected deploy	digest in the running Deployment ≠ the signed baseline digest

Mission 02 — Attack-path hypotheses (conceptual, with ATT&CK + detection)

H1 — Compromised developer identity → CI/CD access → pipeline abuse

- **ATT&CK:** T1078 Valid Accounts · T1098 Account Manipulation (stale identity).
- **Objective:** run pipelines as a trusted dev to push/deploy.
- **Expected logs:** GitLab auth + pipeline audit; SSO/IdP sign-in.
- **Detection:** terminated/stale actor active in automation; run outside release window; new source host. **Blue response:** disable the identity, revoke PATs/SSH keys, rotate CI tokens, audit what it triggered.

H2 — Container supply-chain abuse: CI runner → modified image → registry → deployment

- **ATT&CK:** T1195.002 Supply Chain Compromise (software) · T1610 Deploy Container.
- **Analyze:** image **provenance** (signed? attested?), registry activity, **unexpected digest** for a reused tag, deploy timing, runtime evidence.
- **Detection:** push with no cosign/provenance; digest change vs signed baseline; deploy pulls a non-baseline digest. **Blue response:** enforce admission (only signed digests), freeze the tag, diff the image.

H3 — Kubernetes privilege escalation: compromised workload → excessive RBAC → sensitive resources

- **ATT&CK:** T1078.004 / T1548 · K8s RBAC abuse.
- **High-risk permissions:** cluster-admin on a deploy SA; secrets list/get cluster-wide; pods/exec; create pod in kube-system; hostPath.
- **K8s audit events:** serviceaccount.token.use, secrets list, pod create in kube-system. **Legit vs suspicious:** real deploys patch one namespace's Deployments in-window; cluster-wide secret enumeration + kube-system pods are not deploy behaviour.

Mission 03 — Detection-evasion concept + blue-team catch

Technique — "trusted image, untrusted content": abuse the *legitimate* pipeline so malicious activity inherits the pipeline's trust — same image name/tag, pushed by the real `ci-runner` identity, deployed by the normal `ci-deployer`. Nothing "foreign" appears.

- **Why AV/EDR may miss it:** the image is signed-by-name/trusted-by-policy and runs as an expected workload; there's no malware dropped — it's config/identity abuse (LOTL).
- **Why provenance matters:** without signature + SLSA attestation, "same tag" says nothing — only the **digest + who signed it** proves what actually runs.
- **Artifacts after container deletion:** the **digest** persists in the Deployment spec, ReplicaSet history, kubelet image cache, and the **audit logs**; registry delete removes the blob, not the evidence.
- **How K8s audit exposes it:** the `requestObject` shows the image/digest, the SA, the namespace and verb — the deploy-of-wrong-digest and the secret enumeration are both in the audit stream.
- **Registry + runtime telemetry:** unsigned push, digest mismatch, and (Falco) unexpected egress / shell-in-container / SA-token reads.

● **Blue-team detection chain:** UNEXPECTED IMAGE DIGEST → UNUSUAL/OFF-HOURS PUSH → DEPLOYMENT → NEW SERVICE-ACCOUNT ACTIVITY → CLOUD API ACCESS — encoded in `broken_pipeline_detections.yml` (Sigma-style K8s-audit/registry/cloud rules + a 30-min correlation + Falco runtime rules). No payload/exploit code anywhere.

Mission 04 — Identity & persistence analysis (conceptual, lab)

Area	Finding / control
Impact of excessive CI permissions	deploy SA = cluster-admin → one pipeline compromise = cluster + secrets + cloud. Fix: namespace-scoped Role, no secrets-list, no cloud by default
Stale developer identities	<code>ex.developer</code> terminated 20 Sep, still runs automation. Fix: deprovisioning tied to HR offboarding; audit service identities quarterly
Image provenance verification	require cosign signatures + SLSA attestation; admission controller rejects unsigned/unknown-digest images
Kubernetes RBAC auditing	enumerate bindings for <code>cluster-admin</code> , <code>secrets</code> , <code>pods/exec</code> , <code>escalate</code> , <code>bind</code> ; alert on new ClusterRoleBindings
Cloud credential exposure from workloads	pod workload identity (IRSA/Workload Identity) scoped least-privilege; alert on first-seen external egress from a pod

Senior question — why isn't deleting the malicious container enough after a confirmed K8s compromise? Deleting the container/pod removes one *instance*, not the *cause or the access*. The **identity** (stale dev + the deploy SA/token) still works; the **registry trust** still serves the bad digest (or the Deployment re-pulls it and respawns the pod); the **RBAC** (cluster-admin) is unchanged, so the attacker re-escalates; the **cloud tokens** obtained via workload identity keep working until revoked; and **persistence** (kube-system pod, modified workflow, new bindings) remains. You must rotate identities/tokens, fix RBAC, revoke cloud creds, purge the bad digest with admission enforcement, and hunt persistence — not just `kubectl delete pod`.

Mission 05 — Red-team closure report

Section	Summary (lab/theoretical)
Initial foothold (sim)	trusted-but-stale developer identity active in automation
CI/CD pipeline abuse	off-hours pipeline run outside the release window
Container supply-chain impact	reused tag, new unsigned digest, deployed over the signed baseline
Kubernetes privilege impact	deploy SA cluster-admin → cluster-wide secret enumeration, kube-system pod
Cloud access implications	pod workload identity → AssumeRole → GetSecretValue (prod DB secret)
DFIR evidence	GitLab audit · CI runner logs · registry (push/delete, digest, signature) · K8s audit (deploy digest, SA verbs, namespaces) · cloud audit (AssumeRole/GetSecretValue) · container runtime (Falco)
Detection recommendations	the 6 rules + correlation above; admission signature enforcement; RBAC + egress monitoring
Safe remediation	disable stale identity; namespace-scope the SA; require signed digests at admission; rotate SA tokens + cloud creds; extend K8s audit retention past 7 days

Section	Summary (lab/theoretical)
Business impact	customer data (DB secret → PII) · production availability (deploy manipulation) · software supply chain (future releases) · regulatory (PCI/DPDP for a fintech)

Senior red-team question — A vs B

B is more dangerous. (A) one production container, detected in minutes, is a contained, low-dwell event — the blast radius is one workload and the attacker is evicted fast. (B) owning the **CI/CD trust chain** lets the attacker influence *every future deployment* that ships to production and customers.

- **OPSEC:** A is noisy and short-lived; B hides inside legitimate, expected pipeline activity — signed-by-name, deployed by the normal account — so it blends with trusted operations.
- **Persistence:** A dies with the pod; B re-ships itself every release and can plant backdoors downstream — evicting it means rebuilding trust in the whole pipeline.
- **Blast radius:** A = one container; B = every artifact the pipeline produces, plus the cluster and cloud it can reach, plus customers who receive the builds.
- **Attacker ROI:** A is a single smash-and-grab; B is a durable supply-chain position — highest value, reusable, hardest to fully evict.

Trust the pipeline, verify the artifact, hunt the identity.

MITRE ATT&CK map

T1078 / T1078.004 Valid Accounts · T1098 Account Manipulation · T1195.002 Supply Chain Compromise · T1610 Deploy Container · T1552 Unsecured Credentials · T1530 Data from Cloud Storage · T1537 Transfer to Cloud Account · T1070 (short 7-day audit retention aids evasion)

References: MITRE ATT&CK; Kubernetes Security & Audit Logging docs; GitLab CI/CD security docs; Sigstore/Cosign; OWASP Software Supply Chain Security. LAB ONLY — all data synthetic, no exploit code, no destructive actions.